

Desafio SOA Architecture

Documentação Técnica do Projeto

William Cláudio dos Santos Vaz de Azevedo
Engenharia Informática — ISPTEC
Julho de 2026

Repositório: github.com/williamClaudio1610/Desafio_SOA_Architecture

1. Introdução e Objetivo

Este documento registra, para revisão própria, o trabalho desenvolvido no âmbito do projeto **Desafio SOA Architecture** — um sistema bancário simplificado, construído como exercício prático de arquitetura orientada a serviços (SOA). O sistema é composto por três serviços de domínio (clientes, contas e movimentos), um serviço orquestrador, um API Gateway, e um frontend em React, todos containerizados com Docker.

O objetivo principal não foi construir um sistema bancário completo, mas usar este domínio como pretexto para aprender e aplicar, na prática, conceitos de arquitetura distribuída que normalmente só se estudam em teoria: separação de responsabilidades entre serviços, comunicação entre eles, o papel de um middleware/orquestrador, o papel de um API Gateway, persistência de dados por serviço, e containerização de um sistema multi-serviço com Docker Compose.

2. O que é Arquitetura SOA

SOA (Service-Oriented Architecture) é um estilo arquitetural em que uma aplicação é decomposta num conjunto de serviços independentes, cada um responsável por uma área de negócio bem definida, que comunicam entre si através de interfaces padronizadas — neste projeto, APIs REST sobre HTTP, trocando JSON.

Características principais

- **Baixo acoplamento** — cada serviço desconhece a implementação interna dos outros; só conhece o contrato (endpoints e formato dos dados).
- **Autonomia** — cada serviço tem a sua própria base de dados e pode ser desenvolvido, testado e implantado de forma independente.
- **Responsabilidade única** — cada serviço resolve um problema de negócio específico (clientes, contas, ou movimentos), nunca os três ao mesmo tempo.
- **Interoperabilidade** — comunicação via protocolos e formatos abertos (HTTP + JSON), em vez de chamadas diretas a código ou partilha de base de dados.
- **Composição** — serviços simples podem ser combinados para formar operações de negócio mais complexas. É exatamente este o papel do orquestrador neste projeto.

Como estas características foram aplicadas no projeto

Cada um dos três serviços de domínio (cliente, conta, movimentos) tem a sua própria base de dados PostgreSQL, o seu próprio pom.xml, a sua própria porta, e pode ser corrido isoladamente sem depender dos outros dois estarem no ar. Nenhum deles chama outro diretamente — essa é sempre responsabilidade do orquestrador, nunca dos próprios serviços de domínio entre si.

3. Visão Geral da Arquitetura

O sistema é composto por cinco serviços Spring Boot e um frontend React, organizados em três camadas lógicas:

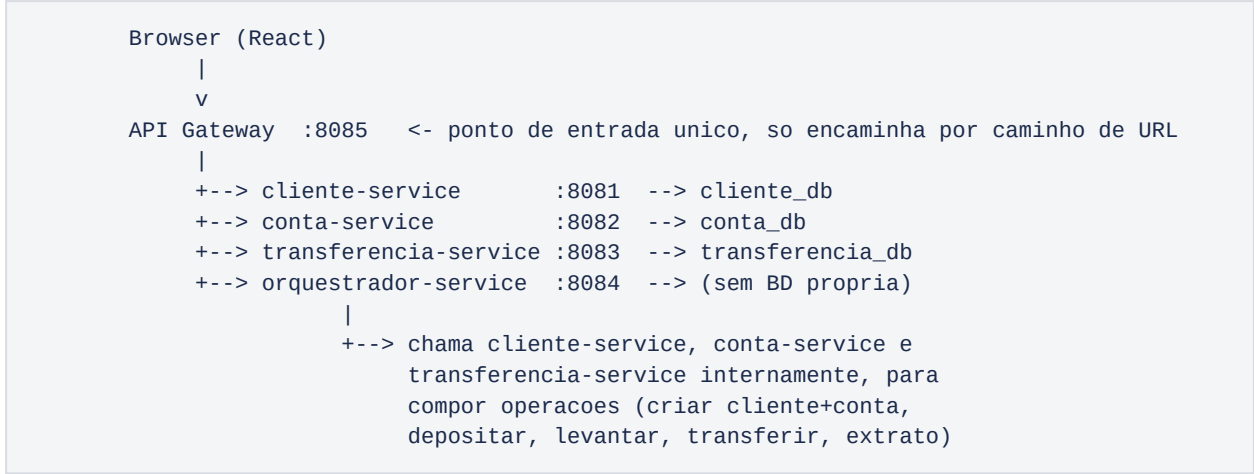


Figura 1 — Camadas da arquitetura

Tabela resumo dos serviços

Serviço	Porta	Responsabilidade	Base de dados
cliente-service	8081	Gestão de clientes (CRUD)	cliente_db
conta-service	8082	Gestão de contas e saldo	conta_db
transferencia-service	8083	Registo de movimentos financeiros	transferencia_db
orquestrador-service	8084	Coordena os 3 serviços acima	— (sem BD própria)
api-gateway	8085	Ponto de entrada único, encaminha por URL	— (sem BD)

4. Os Microserviços de Domínio

4.1 — cliente-service

Responsável apenas por dados de clientes: nome, email, telefone. Não sabe nada sobre contas ou movimentos.

Método	Caminho	Descrição
POST	/api/clientes	Cria um novo cliente
GET	/api/clientes	Lista todos os clientes
GET	/api/clientes/buscar/{id}	Busca um cliente por id
PUT	/api/clientes/{id}	Atualiza um cliente
DELETE	/api/clientes/{id}	Remove um cliente

4.2 — conta-service

Responsável pela conta bancária e pelo saldo. Guarda apenas o `clienteId` do dono da conta (um número, sem chave estrangeira real entre bases de dados diferentes — é o padrão correto em microserviços, já que cada serviço tem a sua própria base).

Método	Caminho	Descrição
POST	/api/contas/criarConta	Cria uma conta para um cliente
GET	/api/contas/listarTodas	Lista todas as contas
GET	/api/contas/conta/{id}	Busca conta por id interno
GET	/api/contas/numero/{numeroConta}	Busca conta pelo número (identificador público)
GET	/api/contas/cliente/{clienteId}	Lista as contas de um cliente
PATCH	/api/contas/atualizarSaldo	Soma/subtrai um valor ao saldo, por número de conta
PATCH	/api/contas/{id}/ativarConta	Reativa uma conta
PATCH	/api/contas/{id}/desativarConta	Desativa uma conta

4.3 — transferencia-service (movimentos)

Regista o histórico de operações financeiras: depósitos, levantamentos e transferências. Cada movimento fica associado ao id interno da conta de origem e, quando aplicável, ao id da conta de destino.

Método	Caminho	Descrição
POST	/api/movimentos/depositar	Regista um depósito
POST	/api/movimentos/levantar	Regista um levantamento
POST	/api/movimentos/transferir	Regista uma transferência
GET	/api/movimentos/conta/{contald}	Lista os movimentos de uma conta

Nota: este serviço apenas regista o movimento — não é ele quem altera o saldo da conta. Essa decisão foi deliberada (ver secção 5) para manter cada serviço com uma única responsabilidade.

5. O Orquestrador (Middleware)

No vocabulário de SOA, **middleware** não é obrigatoriamente um componente novo — é o nome do *papel* de coordenar, traduzir e validar informação entre serviços. Neste projeto, esse papel é desempenhado inteiramente pelo **orquestrador-service** (porta 8084): não existe um quarto serviço "middleware" à parte.

Responsabilidades do orquestrador

- **Sequenciar** chamadas a vários serviços, na ordem correta (ex: criar cliente, depois criar conta com o id devolvido, depois registar o depósito inicial).
- **Traduzir identificadores** entre o número de conta (identificador público, usado pelo cliente da API) e o id interno da base de dados (usado internamente pelo transferencia-service).
- **Validar antes de agir** — confirmar que as contas existem, estão ativas e têm saldo suficiente *antes* de escrever qualquer alteração, evitando ter de desfazer operações a meio.
- **Agregar respostas** de múltiplos serviços numa só resposta ao cliente (ex: o extrato de conta junta dados do cliente, da conta e dos movimentos).
- **Atualizar o saldo** da conta a cada movimento, chamando o conta-service — esta lógica fica no orquestrador, não no transferencia-service, para manter este último simples e focado só em registo histórico.

Exemplo — fluxo de criação de cliente com conta e depósito inicial

```
1. POST /api/orquestrador/onboarding/criar-cliente
2. Orquestrador chama cliente-service -> cria cliente, recebe id
3. Orquestrador chama conta-service -> cria conta com esse
   clienteId e o deposito inicial, recebe numeroConta
4. Orquestrador chama transferencia-service -> regista o
   deposito inicial como movimento (so para historico)
5. Orquestrador devolve: clienteId, nome, numeroConta, saldo
```

Exemplo — validação antes de agir numa transferência

Antes de mexer em qualquer saldo, o orquestrador confirma que a conta de origem existe, está ativa e tem saldo suficiente; depois confirma que a conta de destino existe, está ativa e é diferente da origem. Só depois de as duas validações passarem é que regista o movimento e atualiza os dois saldos. Se qualquer validação falhar, nada foi escrito em lado nenhum — evita a necessidade de uma lógica de compensação/rollback mais complexa.

Classes de suporte

Classe	Papel
ClienteValidador	Confirma que um cliente existe, chamando o cliente-service
ContaValidador	Confirma existência/estado/saldo de uma conta, chamando o conta-service
MovimentoValidador	Valida regras específicas de movimento (ex: conta destino obrigatória só em transferências)

Classe	Papel
GlobalExceptionHandler	Converte exceções de validação em respostas HTTP 400 com mensagem clara, em vez de 500 genéricos

6. O API Gateway

Um **API Gateway** é o único ponto de entrada para os clientes externos (browser, Postman). A sua única responsabilidade é olhar para o caminho do URL de cada pedido e encaminhá-lo, sem alterações, para o serviço correto — não sabe nem precisa de saber nada sobre o significado do negócio.

Gateway vs. Orquestrador/Middleware — diferença aplicada

	API Gateway	Orquestrador (Middleware)
Decide com base em	Caminho do URL, apenas	Conteúdo do pedido e respostas anteriores
Chama outros serviços?	Não — só encaminha	Sim — pode chamar vários, em sequência
Tem lógica de negócio?	Não	Sim — validações, traduções, agregações
Nº de serviços por pedido	Sempre 1	Pode ser vários

Tecnologia utilizada

Implementado com **Spring Cloud Gateway (variante reativa, WebFlux)**, versão do Spring Cloud 2025.0.3, compatível com o Spring Boot 3.5.16 usado em todos os serviços. Ao contrário dos outros 4 serviços (Tomcat/blocking), este corre sobre Netty — mas isso é invisível para quem o usa, já que toda a configuração é feita por ficheiro, sem código Java customizado.

Tabela de rotas configuradas

Prefixo do caminho	Destino
/api/clientes/**	cliente-service (porta 8081)
/api/contas/**	conta-service (porta 8082)
/api/movimentos/**	transferencia-service (porta 8083)
/api/orquestrador/**	orquestrador-service (porta 8084)

CORS

Configurado de forma centralizada no gateway (`globalcors`), permitindo pedidos vindos de `http://localhost:5173` (o frontend React). Por estar centralizado aqui, não é preciso repetir esta configuração em cada um dos 4 serviços internos.

7. Persistência de Dados

Segue-se o padrão **"Database per Service"** — cada serviço de domínio tem a sua própria base de dados PostgreSQL, isolada dos restantes. Nenhum serviço acede diretamente à base de dados de outro.

Serviço	Base de dados	Observações
cliente-service	cliente_db	Tabela clientes
conta-service	conta_db	Tabela contas
transferencia-service	transferencia_db	Tabela movimentos

O esquema das tabelas é gerado automaticamente pelo Hibernate (`spring.jpa.hibernate.ddl-auto=update`), a partir das entidades JPA anotadas em Java — sem necessidade de escrever SQL de criação de tabelas manualmente. Esta opção é adequada para desenvolvimento e para este projeto académico; em produção seria substituída por uma ferramenta de migrations como Flyway, que mantém histórico versionado das alterações de esquema.

8. Containerização com Docker

Todo o sistema — os 5 serviços Spring Boot, o frontend React e a base de dados — sobe com um único comando, através de um `docker-compose.yml` na raiz do repositório.

Estratégia geral

- Um único container PostgreSQL, com as 3 bases de dados criadas automaticamente no arranque através de um script SQL de inicialização.
- Um container por cada um dos 5 serviços Spring Boot / gateway, cada um com o seu próprio Dockerfile.
- Um container nginx a servir os ficheiros estáticos do frontend React já compilado.
- Todos os containers na mesma rede interna do Docker Compose, comunicando entre si pelo *nome do serviço* em vez de `localhost`.
- Nenhum ficheiro de configuração local (`application.properties/yml`) foi alterado — a configuração específica do Docker entra inteiramente por variáveis de ambiente definidas no próprio `docker-compose.yml`, preservando o fluxo de desenvolvimento local no IntelliJ sem alterações.

Dockerfile dos serviços Spring Boot (build em duas etapas)

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Figura 2 — Dockerfile típico de um serviço Spring Boot (build + runtime separados)

Dockerfile do frontend (nginx a servir ficheiros estáticos)

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

Figura 3 — Dockerfile do frontend (build Vite + nginx)

Rede interna e variáveis de ambiente

Dentro da rede do Docker Compose, os serviços não usam `localhost` para se encontrarem — usam o nome do serviço, que o Docker resolve automaticamente:

```
SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/cliente_db
CLIENTE_SERVICE_URL: http://cliente-service:8081/api/clientes
CONTA_SERVICE_URL: http://conta-service:8082/api/contas
```

Figura 4 — Exemplos de variáveis de ambiente injetadas no docker-compose.yml

Arranque ordenado (healthcheck)

O container do PostgreSQL tem um healthcheck (pg_isready), e os 3 serviços de domínio só arrancam depois de a base de dados estar mesmo pronta a aceitar ligações (condition: service_healthy) — evita falhas de ligação no primeiro arranque, quando o Postgres ainda não tinha terminado de inicializar.

Persistência de dados entre reinícios

Os dados do PostgreSQL do Docker são guardados num volume nomeado (postgres_data), que sobrevive a docker compose down e a reinícios do container. Só é apagado com o comando explícito docker compose down -v.

Nota: o PostgreSQL do Docker é inteiramente separado do PostgreSQL instalado localmente no Windows (usado durante o desenvolvimento no IntelliJ) — mesmo tendo bases de dados com o mesmo nome, vivem em portas e volumes de armazenamento diferentes (porta 5433 para o Docker, mapeada para a 5432 interna do container; porta 5432 para a instalação nativa do Windows).

9. Frontend

Interface simples de demonstração, construída com **React** e **Vite** (variante JavaScript, sem TypeScript nem framework de routing completo — desnecessário para um painel de demonstração com apenas 3 ecrãs).

Estrutura

- `src/api.js` — única camada que fala com o backend; todas as chamadas passam exclusivamente pelo API Gateway (porta 8085), nunca diretamente para um serviço individual.
- `src/App.jsx` — três ecrãs (criar cliente, realizar movimento, ver extrato), com navegação simples por estado local (`useState`), sem necessidade de biblioteca de rotas.
- Tratamento de erro uniforme: as mensagens de erro devolvidas pelo `GlobalExceptionHandler` do orquestrador são mostradas diretamente ao utilizador, sem reescrita.

Comunicação com o backend

```
const BASE_URL = 'http://localhost:8085'; // gateway, nunca muda

export async function criarClienteComConta(dados) {
  const res = await fetch(
    `${BASE_URL}/api/orquestrador/onboarding/criar-cliente`,
    { method: 'POST', headers: {'Content-Type': 'application/json'},
      body: JSON.stringify(dados) });
  return res.json();
}
```

Figura 5 — Excerto de `api.js`: o frontend só conhece o endereço do gateway

10. Stack Tecnológica

Backend

Tecnologia	Utilização no projeto
Java 21	Linguagem base de todos os serviços
Spring Boot 3.5.16	Framework principal dos 5 serviços
Spring Web (MVC)	Camada REST dos 4 serviços Tomcat (cliente, conta, movimentos, orquestrador)
Spring Cloud Gateway (WebFlux) 2025.0.3	API Gateway
Spring Data JPA / Hibernate	Persistência e mapeamento objeto-relacional
PostgreSQL 17	Base de dados relacional, uma instância por serviço de domínio
RestTemplate	Comunicação HTTP síncrona entre orquestrador e serviços de domínio
Lombok	Redução de código repetitivo (getters/setters)
SpringDoc OpenAPI (Swagger)	Documentação e teste interativo dos endpoints
Maven	Gestão de dependências e build

Frontend

Tecnologia	Utilização no projeto
React 18+	Biblioteca de interface
Vite	Ferramenta de build e servidor de desenvolvimento
Fetch API	Chamadas HTTP ao gateway

Infraestrutura

Tecnologia	Utilização no projeto
Docker / Docker Compose	Containerização e orquestração local de todos os serviços
nginx	Servidor de ficheiros estáticos do frontend, dentro do container
Git / GitHub	Controlo de versão, repositório público do projeto

11. Como Executar o Projeto

Opção A — Localmente, via IntelliJ

- Garantir que o PostgreSQL local está a correr e que as bases `cliente_db`, `conta_db` e `transferencia_db` existem.
- Correr, por esta ordem, os 5 módulos: `cliente-service`, `conta-service`, `transferencia-service`, `orquestrador-service`, `api-gateway`.
- Correr o frontend com `npm run dev`, dentro da pasta do projeto React.
- Aceder à aplicação em `http://localhost:5173`.

Opção B — Via Docker Compose (recomendado para demonstração)

```
docker compose up --build
```

Constrói as 6 imagens e sobe todos os containers

Depois de todos os containers arrancarem, a aplicação fica acessível em `http://localhost:5173`, com todo o backend por trás já comunicando internamente pela rede do Docker.

```
docker compose down      # para os containers, mantém os dados
docker compose down -v   # para os containers e apaga também os dados
```

12. Principais Desafios e Aprendizagens

Alguns dos problemas mais instrutivos encontrados durante o desenvolvimento, porque expuseram exatamente os pontos de fricção típicos de sistemas distribuídos:

- **Contratos JSON divergentes entre serviços** — o mesmo conceito (ex: tipo de conta) foi nomeado de forma diferente em dois serviços (`tipoConta` vs `tipoconta`), fazendo com que um campo chegasse sempre nulo do outro lado sem erro explícito. Este tipo de bug só aparece ao testar a integração real entre serviços, nunca isoladamente.
- **Repositories mal implementados** — uma interface Spring Data JPA foi acidentalmente convertida numa classe concreta pelo autocomplete da IDE, impedindo o Spring de gerar a implementação automática.
- **Codificação de ficheiros** — um ficheiro de configuração gravado em ISO-8859-1 (em vez de UTF-8) compilava sem problemas no Windows/IntelliJ, mas falhava dentro do container Docker, cujo ambiente Linux não assume a mesma codificação por omissão.
- **Ambiguidade de identificadores** — decidir se as chamadas entre serviços deveriam usar o id interno da base de dados ou o número de conta (identificador público) exigiu tornar explícita, num único ponto do orquestrador, a tradução entre os dois — em vez de deixar essa decisão implícita e inconsistente espalhada pelo código.
- **Diagnóstico de erros silenciosos** — respostas 500 sem mensagem nenhuma tornavam a depuração lenta; a introdução de um `GlobalExceptionHandler` no orquestrador tornou os erros de validação imediatamente legíveis.

13. Possíveis Evoluções Futuras

- **Flyway** — substituir `ddl-auto=update` por migrations versionadas, com histórico auditável de alterações ao esquema.
- **RabbitMQ** — mover a comunicação entre orquestrador e serviços de domínio para mensagens assíncronas, tornando o sistema mais resiliente a indisponibilidade temporária de um serviço.
- **Autenticação/JWT** — centralizar autenticação no API Gateway, em vez de deixar todos os endpoints publicamente acessíveis.
- **Testes automatizados** — cobertura de testes unitários e de integração para os serviços e para o orquestrador.
- **Padrão Saga** — para operações que envolvam múltiplas escritas distribuídas com necessidade real de compensação, em vez do atual modelo de validar-tudo-antes-de-escrever.

Documento gerado para revisão pessoal do percurso de aprendizagem do projeto Desafio SOA Architecture.